설비 고장을 예측하는 AI 데이터 분석

설비 상태 변화 패턴 분석

상태 변화 추적

posco

CONTENTS

목차

01 상태 변화의 관점 — 값이 아닌 변화를 보는 이유

02 이동통계 추적 — 이동평균과 이동표준편차로 흐름 읽기

03 변화량과 변화점 — diff·이동평균 변화량·잔차 기반 탐지

posco

posco

01

상태 변화의 관점

값이 아닌 변화를 보는 이유와 전기 Panel 데이터 준비

변화의 흐름을 보는 이유

절대값이 아닌 평소와 비교한 변화를 추적해 신호를 읽음

01

설비 고장은 갑자기가 아니라 **데이터의 흐름** 속에 신호를 남김

02

같은 센서값도 설비마다 정상일 수도 이상일 수도 있음

03

절대값이 아니라 **평소와 비교한 변화**를 추적

posco

값이 아닌 변화를 보는 관점

한 점이 아니라 여러 점을 줄로 이어 방향을 파악

01 · 절대값의 함정

센서마다
정상 범위가 다름

같은 640도라도 어느 엔진엔 정상, 어느 엔진엔 상승 신호 — 값 하나로는 판단 불가

02 · 흐름 관점

여러 점을
줄로 이어 방향

시간 순서로 정렬된 시계열 데이터로 올라가는지 내려가는지 흐름 확인

03 · 의미있는 변화

노이즈 걸러
진짜 변화만 추적

센서의 작은 흔들림까지 변화로 보면 거짓 경보 — 노이즈는 걸러야 함

posco

설비 상태 변화 추적이란

센서 데이터가 시간에 따라 어떤 방향·속도로 달라지는지 따라가며 설비 건강 읽기

DEFINITION

상태 변화 추적= 센서 데이터가 시간에 따라 달라지는 흐름을 놓치지 않고 따라가는 작업

상태

얼마나 건강한가

변화

시간에 따라 달라짐

추적

놓치지 않고 따라감

설비는 시간이 지날수록 부품이 닳으므로 **대부분 천천히 나빠지는 방향**

상태 변화 추적의 목적

예지보전(PdM) — 고장 전에 미리 알아채 계획적으로 대비

01 · 큰 목표

예지보전(PdM) — 고장난 뒤 고치는 게 아니라 고장 전에 미리 알아채 대비

02 · 역할

변화 추적은 이상 탐지와 고장 예측의 **기초 재료**

03 · 해석

결과 해석에는 설비 지식이 필요 — **도메인이 자산**

posco

설비 상태가 데이터에 남기는 흔적

정상은 잔잔, 나빠지면 추세와 변동성 두 흔적이 함께 나타남

01 · 정상

일정한 값 주변에서 작게 흔들림

건강한 설비의 모습 — 자연스러운 측정 오차에 의한 작은 흔들림

02 · 추세

값이 한쪽으로 서서히 쏠림

부품 마모·효율 저하로 평균이 한 방향으로 천천히 이동

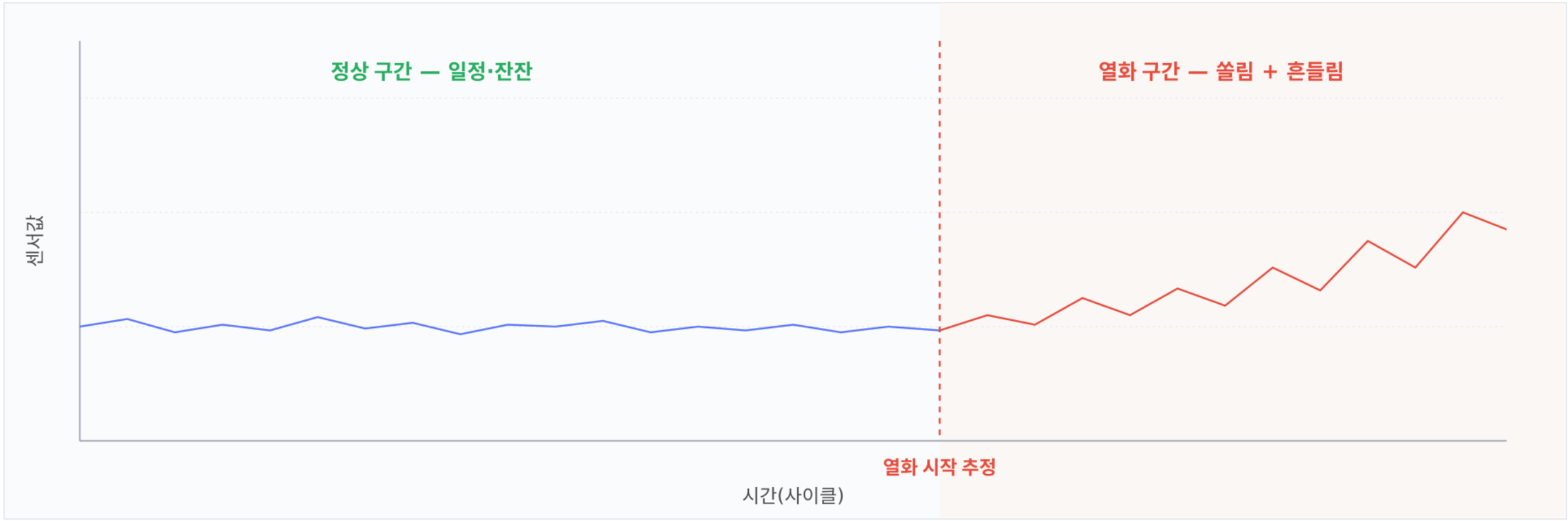
03 · 변동성

흔들림 폭이 점점 커짐

0.3도 흔들리던 게 1~2도씩 출렁이기 시작 — 설비가 불안정해짐

정상에서 열화로 — 데이터 변화 모습

앞쪽은 일정·잔잔, 뒤쪽은 쏠림과 흔들림 동시 증가



온도는 오르지만 압력은 내려가기도 함 — 방향보다 평소 패턴에서 벗어나는가를 봄

posco

전기 Panel 데이터의 구조

KAMP 전기 Panel 데이터 — 설비번호 · 시퀀스 · 전류 신호로 구성

01 · 설비번호

설비 식별 번호

어느 설비에서 측정한 기록인지 구분 — 1번 설비인지 2번 설비인지

값 범위: 1 ~ 5

02 · 시퀀스

시간 순서 번호

설비별 측정 순서를 나타내는 번호 — 각 설비에서 250개 시점 측정

설비마다 시퀀스 1~250의 측정 기록

03 · 전류 센서 5개

전류 신호를 센서별 기록

컨베이어·회전A/B·펌프A/B 전류 등 5개 신호를 사용

예: 컨베이어전류, 펌프A전류

전기 Panel 데이터의 특징

설비 5대의 전류 신호를 각 250시점씩 기록 — 전체 1,250행

01 · 시간 단위

시퀀스는 각 설비 안에서 측정 순서를 나타냄

02 · 측정 범위

설비별 250개 시점의 전류 데이터를 수록

03 · 별칭

설비번호·시퀀스와 전류 센서 5개 컬럼으로 구성

posco

한 설비씩 추출하는 이유

여러 설비를 그냥 이으면 경계에서 가짜 변화 발생

여러 설비의 기록을 그대로 이으면 시퀀스 흐름이 섞임

1번 설비의 마지막 시점 다음에 2번 설비의 첫 시점이 오면 설비 경계가 변화처럼 보임



실제 변화가 아니라 설비가 바뀐 것 — 반드시 한 설비씩 분리해 분석

posco

단일 설비 분석 3단계

설비 한 대를 먼저 골라낸 다음 시퀀스 순으로 정렬 — 분석의 첫 단추

STEP 1

조건 필터링

설비번호가 1인 행만 골라내기. 앞 파트에서 배운 조건 추출을 그대로 사용

```
df["설비번호"] == 1
```

→

STEP 2

시간 순 정렬

시퀀스 기준으로 정렬. 시계열은 순서가 생명 — 정렬해두면 안전

```
sort_values("시퀀스")
```

→

STEP 3

인덱스 재설정

원래 인덱스 대신 0부터 다시 매기기. 작은 습관이 실수를 줄임

```
reset_index(drop=True)
```

규칙은 단순 — 상태 변화 추적 시 항상 설비 한 대를 먼저 골라낸 다음 분석

실습 1. 데이터 불러오기와 단일 설비 신호 그리기

한 설비를 골라 전류 신호를 시간순으로 그리기

목표

데이터를 불러와 구조를 확인하고 한 설비의 전류 신호를 시간순으로 그리기

단계

· 파일을 불러와 크기와 설비번호 목록 확인

· 2번 설비만 골라 시퀀스 순으로 정렬

· 컨베이어전류를 시간순 선 그래프로 그리기

예상 결과

전체 1,250행·설비 5대, 2번 설비 250행; 컨베이어전류 곡선이 그려짐

posco

실습 1. 데이터 불러오기와 단일 설비 신호 그리기 정답

한 설비를 골라 전류 신호를 시간순으로 그리기

CODE · 실습 1

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('../data/23_전기Panel.csv')
print(df.shape, df['설비번호'].unique())      # (1250, 7) [1 2 3 4 5]
u2 = df[df['설비번호'] == 2].copy().sort_values('시퀀스').reset_index(drop=True)
print(u2.shape)                                    # (250, 7)
plt.plot(u2['시퀀스'], u2['컨베이어전류']); plt.title('설비2 - 컨베이어전류'); plt.show()
```

02

이동통계 추적

rolling window · 이동평균 · 이동표준편차로 흐름 읽기

posco

rolling window로 데이터 보기

일정 크기의 창을 데이터 위에 놓고 한 칸씩 굴리며 계산

01 · 발상

흔들림 속 진짜 흐름을 보려면 값 하나가 아니라 **여러 값을 묶어** 대표값을 구함

02 · 동작 원리

rolling window — 일정 크기의 창을 데이터 위에 놓고 **한 칸씩 굴리며** 계산

03 · 판다스 표현

`.rolling(window=크기)`로 창을 만들고 `.mean()`이나 `.std()`를 이어 붙임

posco

rolling window 작동 순서

창 올리기 → 계산 → 한 칸 굴려 다시 계산

STEP 1

창 올리기

데이터 위에 정해진 크기의 창을 올림. 창 크기가 10이면 연속된 값 10개가 들어옴

→

STEP 2

계산

창 안의 값들로 평균이나 표준편차를 계산. 결과는 보통 창의 끝자리에 적힘

→

STEP 3

한 칸 굴리기

창을 한 칸 오른쪽으로 굴려 다시 계산. 미끄러지듯 굴러간다 해서 rolling

mean을 붙이면 이동평균, std를 붙이면 이동표준편차

이동평균으로 추세 읽기

튀는 값을 주변과 섞어 곡선을 평탄하게 만듦

01 · 문제

센서는 매 순간 측정 오차로 튀어서 **추세가 헷갈림**

02 · 처방

이동평균은 창 크기만큼 평균 내어 튀는 값을 **주변과 섞어 평탄하게**

03 · 효과

흔들림은 줄고 전체 흐름만 남아 **우상향·우하향 추세가 또렷**

이동평균이 보여주는 것

노이즈에 숨은 약한 추세를 끄집어내는 도구이자 변화점의 기준선

01 · 추세 방향

이동평균선이 **우상향이면 상승 추세**, 우하향이면 하강 추세

02 · 약한 신호 포착

노이즈에 숨은 **약한 추세를 끄집어내는** 도구

03 · 기준선 역할

뒤에서 변화점을 찾을 때 **기준선** 역할

posco

window 크기의 영향

창 크기는 민감함과 부드러움 사이의 균형을 고르는 일

01 · 작은 창 (예: 3)

민감하지만 시끄러움

변화에 빠르게 반응하지만 흔들림이 남아 곡선이 들쭉날쭉

가까운 3개만 평균 → 노이즈 못 거름

02 · 큰 창 (예: 30)

매끄럽지만 둔감함

곡선은 깔끔하지만 과거 값에 묻혀 변화에 한참 뒤늦게 반응

30개 평균 → 변화 놓칠 수 있음

어느 쪽이 좋다기보다 무엇을 보려는지에 따라 다름

window 크기 고르는 기준

잡고 싶은 변화의 시간 폭과 창 크기를 맞추는 게 요령

01 · 변화의 폭 맞추기

잡고 싶은 변화가 **몇 개의 데이터에 걸쳐** 일어나는지 생각해 비슷하게 맞춤

02 · 무난한 출발점

수백 사이클이면 창 크기 **10에서 30** 사이가 무난

03 · 피해야 할 선택

데이터 길이의 절반에 가까운 큰 창, 노이즈를 못 거르는 **1~2짜리 창**

rolling 실무 주의점

초보자가 꼭 만나는 함정 — NaN, min_periods, 결측치 순서

01 · 앞부분 NaN

창이 안 채워진 앞쪽은 빈 값

오류가 아닌 정상 — 창 크기 10이면 앞 9칸은 값이 모자라 NaN

02 · min_periods

최소 개수 지정으로 앞부분도 채우기

min_periods=1로 주면 값 1개만 있어도 계산 — 다만 앞쪽은 덜 안정적

03 · 결측 먼저

중간 결측은 먼저 처리

그 구간 결과가 비어 곡선이 끊김 — 이동평균 전에 결측 처리 필수

결과 저장과 원본 보존

원본 옆에 새 관점의 값을 더하는 것 — 원본 훼손이 아니라 새 열 추가

01 · 저장 위치

이동평균 결과는 원본과 같은 길이의 새 값 — 보통 새 **열로 추가**해 보관

02 · 열 이름

열 이름에 창 크기를 적으면(`ma10`, `ma30`) 나중에 헷갈리지 않음

03 · 원본 보존

이동평균을 구해도 **원본은 그대로** — 옆에 새 관점의 값을 더하는 것

posco

원본과 이동평균 겹쳐 그리기

실제 흔들림과 전체 흐름을 동시에 보는 표준 시각화 틀

STEP 1

이동평균 계산

이동평균을 계산해 새 열로 저장. 원본과 같은 길이의 새 시리즈

```
.rolling(10).mean()
```

STEP 2

그래프 영역

그래프 영역을 만들고 가로축을 시간(사이클)으로 지정

```
plt.figure()
```

STEP 3

plot 두 번 호출

원본을 옅게 먼저, 그 위에 이동평균선을 진하게 덧그리고 범례 표시

```
plot(raw) + plot(ma) + legend()
```

어떤 센서, 어떤 설비든 똑같이 쓰는 표준 패턴

posco

겹쳐 그린 그래프 읽기

방향 · 변곡점 · 벗어남 — 세 관점으로 해석

01 · 방향

이동평균선이 어느 쪽으로 가나

우상향·우하향·평평한지로 추세 파악 — 기울기가 곧 쏠림의 정도

02 · 변곡점

평평하다 기울기 시작하는 지점

설비 상태가 달라지기 시작한 시점일 가능성

03 · 벗어남

원본이 평균선에서 멀어질수록

원본이 평균선에서 점점 멀리 출렁이면 불안정해지는 신호

코드로 그림을 만드는 건 절반, 나머지는 설비의 언어로 해석

posco

실습 2. 단일 센서 이동평균 추적

창 크기 10 이동평균을 원본에 겹쳐 추세 읽기

목표

창 크기 10 이동평균을 구해 원본 위에 겹쳐 추세를 읽기

단계

· 센서에 창 크기 10 이동평균을 새 열로 담기

· 원본을 옅게, 이동평균을 진하게 겹쳐 그리기

· 이동평균선이 추세를 드러냄을 확인

예상 결과

들쭉날쭉한 원본 위로 매끈한 이동평균 추세선

posco

실습 2. 단일 센서 이동평균 추적 정답

창 크기 10 이동평균을 원본에 겹쳐 추세 읽기

CODE · 실습 2

```
u2['ma10'] = u2['컨베이어전류'].rolling(window=10).mean()
print(u2['ma10'].head(11).round(4).tolist())      # 앞 9개 nan
plt.plot(u2['시퀀스'], u2['컨베이어전류'], color='lightgray', label='raw')
plt.plot(u2['시퀀스'], u2['ma10'], color='red', label='MA(10)')
plt.legend(); plt.show()
```

실습 3. 창 크기 3종 비교

창 크기를 바꿔 매끈함과 반응성의 맞교환 비교

목표

창 크기를 5·10·30으로 바꿔 매끈함과 반응성의 맞교환을 비교

단계

· 세 가지 창 크기로 이동평균을 각각 구하기

· 세 이동평균을 한 그래프에 겹쳐 그리기

· 창이 클수록 매끄럽지만 반응이 느려짐을 확인

예상 결과

창이 클수록 곡선이 매끄럽고 앞쪽 빈 구간이 늘어남

posco

실습 3. 창 크기 3종 비교 정답

창 크기를 바꿔 매끈함과 반응성의 맞교환 비교

CODE · 실습 3

```
for w in [5, 10, 30]:
    u2[f'ma{w}'] = u2['컨베이어전류'].rolling(window=w).mean()
    plt.plot(u2['시퀀스'], u2[f'ma{w}'], label=f'MA({w})')
plt.legend(); plt.title('window comparison'); plt.show()
```

posco

이동표준편차로 변동성 읽기

이동평균이 어느 쪽으로 가나라면, 이번엔 얼마나 흔들리나

01 · 문제

추세는 이동평균으로 봤고, 흔들림이 커지는 건 **표준편차**로 봄

02 · 정의

표준편차는 값들이 **평균에서 얼마나 흩어졌는지**를 나타내는 숫자

03 · 판다스 표현

같은 창에 `.mean()` 대신 **`.std()`**를 붙이면 이동표준편차

변동성이 알려주는 신호

추세보다 변동성이 먼저 커지는 경우도 있음 — 둘을 항상 함께 봄

01 · 안정 vs 불안정

이동표준편차가 **낮으면 안정**, 솟으면 불안정해진 신호

02 · 결정적 차이

떨림의 크기를 본다는 점이 이동평균과의 차이

03 · 조기 신호

추세보다 변동성이 **먼저 커지는** 경우도 있음

추세 변화 vs 변동성 변화

중심 이동 vs 흩어짐 증가 — 포착 도구가 다름

관점	추세 변화	변동성 변화
무엇이 변하나	**값의 중심이 이동**	**값의 흩어짐이 커짐**
예시	평균 642 → 646도로 상승	0.3도 흔들림 → 1.5도 출렁
반영하는 것	부품의 점진적 마모	설비가 불안정해지는 모습
포착 도구	**이동평균 (기울기)**	**이동표준편차 (높이)**

두 변화를 함께 봐야 하는 이유

하나만 보면 절반만 보는 셈 — 평균 상승과 변동성 증가는 다른 정보

01 · 고장 종류 차이

어떤 고장은 추세부터, 어떤 고장은 변동성부터 변함 — **하나만 보면 절반만**

02 · 이상적 추적

이동평균(중심)과 이동표준편차(떨림)를 **함께** 보는 것

03 · 리포트 작성

평균 상승과 변동성 증가는 **다른 정보**로 구분해 표현

posco

이동평균과 이동표준편차 계산

초반·후반 구간의 이동표준편차 평균으로 변동성 비교

STEP 1

두 통계 계산

rolling(10) 평균과 표준편차를 각각 ma10, std10 열에 저장

```
.rolling(10).mean() / .std()
```

STEP 2

초반 구간

std10.iloc[10:50].mean()으로 초반 이동표준편차 평균 계산

```
early = std10.iloc[10:50].mean()
```

STEP 3

후반 구간 비교

std10.iloc[150:].mean()을 계산해 초반 평균과 비교

```
late = std10.iloc[150:].mean()
```

현재 데이터: 초반 0.011, 후반 0.000

초반·후반 변동성 평균 비교

현재 데이터는 후반 변동성이 낮음 — 계산 결과로 확인

초반 구간

std10 평균은 0.011

후반 구간

std10 평균은 0.000

해석

이 데이터에서는 후반 변동성이 커지지 않음 — 결과에 맞춰 해석

실습 4. 이동평균·이동표준편차 동시 추적

이동평균·이동표준편차로 추세와 변동성 추적

목표

창 크기 10으로 이동평균과 이동표준편차를 계산해 초반과 후반 변동성을 비교

단계

· 창 크기 10으로 이동평균과 이동표준편차를 구하기

· 이동평균과 이동표준편차는 계산해 열에 저장

· 초반과 후반의 변동성 평균을 비교

예상 결과

초반/후반 변동성은 0.011 / 0.000으로 계산됨

posco

실습 4. 이동평균·이동표준편차 동시 추적 정답

추세와 변동성을 함께 계산해 동시에 추적

CODE · 실습 4

```
u2['ma10'] = u2['컨베이어전류'].rolling(10).mean()
u2['std10'] = u2['컨베이어전류'].rolling(10).std()
early = u2['std10'].iloc[10:50].mean()
late = u2['std10'].iloc[150:].mean()
print('초반/후반 변동성:', round(early, 4), round(late, 4))   # 0.011 / 0.0
```

03

변화량과 변화점

diff · 이동평균 변화량 · 잔차 기반 변화점 탐지

변화량(diff)

한 발짝 사이에 얼마나 변했나 — 빼기만 하면 됨

01 · 정의

변화량은 지금 값에서 **직전 값을 뺀** 것

02 · 무엇을 보는가

값이 얼마인가가 아니라 한 걸음의 변화 **속도**를 봄

03 · 판다스 표현

`.diff()` 한 줄 — 맨 첫 값은 뺄 직전 값이 없어 NaN

변화량이 알려주는 것

부호로 방향, 절대값으로 속도 — 자동차 가속 페달과 같은 발상

01 · 양수 (+)

값이 직전보다 올라감

예: 643 → 645 (변화량 +2). 상승 신호로 해석 가능

02 · 음수 (−)

값이 직전보다 내려감

예: 645 → 643 (변화량 −2). 하강 신호로 해석 가능

03 · 0 근처

값이 거의 그대로 유지

급변이 일어나면 0에서 멀리 크게 튐 — 급변 후보

이동평균 후 변화량

이동평균을 적용한 뒤 차분하면 원본의 급격한 흔들림이 완화됨

01 · 원본 변화량

diff()는 연속한 시점 사이 전류값의 차이를 계산

02 · 평활화

이동평균을 먼저 구한 뒤 diff()를 적용하면 변화량이 더 완만해짐

03 · 이동평균 변화량

ma_diff는 이동평균의 연속 시점 차이 — 원본 차분보다 흔들림이 적음

원본 변화량과 이동평균 변화량

연속 시점 차이 vs 평활화된 차이 — 분석 목적에 맞게 구분

관점	원본 변화량 (diff)	이동평균 변화량 (ma_diff)
의미	**절대적인 차이**	**이동평균의 연속 차이**
묻는 질문	몇이나 변했나	추세가 얼마나 변했나
언제 유리한가	크기가 비슷한 센서끼리	**완만한 추세 변화 확인**
주의점	노이즈에 민감	**급격한 변화가 완만해질 수 있음**

posco

변화량으로 시점 간 급변 확인

첫 변화량은 NaN — 이후 diff와 이동평균 차분을 함께 확인

01 · 급변 후보 찾기

변화량 그래프에서 **평소보다 크게 튄** 봉우리를 찾으면 급변 후보

02 · 기준선 — 표준편차의 몇 배

ma_diff는 이동평균의 연속 시점 차이로 원본 diff보다 노이즈가 적음

03 · 해석 주의

변화량은 급변 후보 — 설비 맥락과 원본 신호를 함께 확인

느린 변화와 빠른 변화

변화의 성격에 맞는 도구 선택 — 도구는 후보를 짚고, 사람이 확인

01 · 느린 변화

천천히 진행되는 열화

이동평균으로 포착 — 부드러운 곡선 위에서 추세가 보임

02 · 빠른 변화

순간적인 급변

부드러운 이동평균에선 묻혀 — 변화량(diff)으로 포착

03 · 후보 확인

봉우리 발견 후 사람이 직접 점검

진짜 이상인지 센서 오작동인지 — 전후 원본 직접 확인

posco

실습 5. 변화량 계산과 이동평균 변화량 확인

변화량과 이동평균 변화량으로 신호의 변화를 확인

목표

연속 시점의 변화량과 이동평균 후 변화량을 계산해 비교

단계

· 컨베이어전류의 직전 대비 변화량을 구하기

· 첫 값이 비교 대상이 없어 비는 이유 확인

· 이동평균 후 변화량(ma_diff)도 계산하기

예상 결과

변화량 첫 네 값은 [nan, 0.0, 0.0, 0.0]; 최대 변화량은 0.2602

posco

실습 5. 변화량 계산과 이동평균 변화량 확인 정답

변화량과 이동평균 변화량을 계산해 신호 변화 확인

CODE · 실습 5

```
u2['diff'] = u2['컨베이어전류'].diff()
print('변화량:', u2['diff'].head(4).round(4).tolist())  # [nan, 0.0, 0.0, 0.0]
idx = u2['diff'].abs().idxmax()
print('최대 변화량:', round(u2['diff'].abs().max(), 4))  # 0.2602
u2['ma_diff'] = u2['ma10'].diff()
```

변화점(change point)이란

흐름이 그 전과 뚜렷하게 달라지기 시작하는 지점 — 상태 변화 시점

01 · 정의

변화점은 데이터의 흐름이 그 전과 **뚜렷하게 달라지기** 시작하는 지점

02 · 어디서 보이나

평평하던 이동평균선이 기울기 시작하거나 **변동성이 갑자기 커지는 곳**

03 · 설비 의미

설비에서 변화점은 곧 **상태가 달라지기 시작한 시점**

변화점의 성격

확정 아닌 의심 후보 — 여기를 더 살펴보라는 표지판

01 · 값이 아닌 흐름

변화점은 값이 큰 곳이 아니라 **흐름이 바뀌는** 곳

02 · 의심 후보

데이터가 짚는 변화점은 확정이 아닌 **의심 후보**

03 · 여러 개 가능

설비가 여러 단계로 나빠지면 변화점이 **여러 개**

posco

잔차 기반 변화점 탐지

배운 도구만으로 변화점을 잡는 단순하고 강력한 로직

STEP 1

기준선 마련

이동평균을 구해 평소 이 정도일 거라는 기준선으로 삼음

```
roll_mean = s.rolling(w).mean()
```

→

STEP 2

잔차 계산

원본에서 이동평균을 뺀 잔차 — 기준선에서 얼마나 떨어졌나

```
residual = s − roll_mean
```

→

STEP 3

임계값 초과 후보

잔차의 절댓값이 잔차 표준편차의 2배 넘으면 변화점 후보

```
|resid| > 2 × resid.std()
```

한 문장 — 평소 흔들림의 2배 넘게 튄 곳을 변화점으로 봄

변화점 탐지의 두 손잡이

창 크기와 임계값 배수 — 느슨함과 엄격함의 균형점 찾기

01 · 공정한 자동 기준

절대값이 아닌 **평소 대비 벗어남**으로 판단 — 센서마다 기준 자동 조정

02 · 두 손잡이

이동평균 **창 크기**, 임계값 **배수(2배·3배)**

03 · 균형점 찾기

느슨하면 후보가 많고 엄격하면 적음 — **목적에 맞게 조절**

CUSUM 누적합 맛보기

한 번에 크게 튀진 않지만 작은 차이가 같은 방향으로 계속 쌓이는 경우

01 · 발상

CUSUM은 작은 차이를 **계속 더해 쌓아** 변화를 잡는 방법

02 · 잔차 방식과의 차이

잔차는 한 번에 크게 튄 것, CUSUM은 **조금씩 쌓인** 것

03 · 학습 범위

이 과정에서 직접 구현하지 않고 **발상만 이해**

실습 6. 단순 변화점 탐지와 구간 표시

잔차와 임계값으로 변화점을 찾아 세로선으로 표시

목표

잔차와 임계값으로 변화점을 찾고 그래프에 세로선으로 표시

단계

· 원본에서 이동평균을 뺀 잔차를 구하기

· 잔차 표준편차의 두 배를 임계값으로 정해 변화점 표시

· 변화점 위치를 세로선으로 그래프에 표시

예상 결과

임계값 0.0536, 변화점 12개

posco

실습 6. 단순 변화점 탐지와 구간 표시 정답

잔차와 임계값으로 변화점을 찾아 세로선으로 표시

CODE · 실습 6

```
roll = u2['컨베이어전류'].rolling(10).mean()
resid = u2['컨베이어전류'] - roll
thr = 2 * resid.std()
u2['is_change'] = resid.abs() > thr
print('임계값:', round(thr, 4), '변화점:', int(u2['is_change'].sum()))   # 0.0536 / 12
for c in u2.loc[u2['is_change'], '시퀀스']:
    plt.axvline(c, color='orange', linestyle='--', alpha=0.7)
plt.plot(u2['시퀀스'], u2['컨베이어전류'], color='lightgray'); plt.savefig('실습6_change.png'); plt.close()
```

posco

실습 7. window·기준값 바꿔 결과 비교

창 크기와 임계 배수로 변화점 개수 변화 비교

목표

창 크기와 임계 배수를 바꿔 변화점 개수가 어떻게 달라지는지 비교

단계

· 잔차 기반 변화점 개수를 세는 함수를 만들기

· 창 크기를 바꿔 가며 변화점 개수 비교

· 임계 배수를 바꿔 가며 변화점 개수 비교

예상 결과

창이 커지거나 배수가 작아질수록 변화점 개수가 달라짐

posco

실습 7. window·기준값 바꿔 결과 비교 정답

창 크기와 임계 배수로 변화점 개수 변화 비교

CODE · 실습 7

```
def count_changes(series, window, k):
    r = series - series.rolling(window).mean()
    return int((r.abs() > k * r.std()).sum())
s = u2['컨베이어전류']
print({w: count_changes(s, w, 2) for w in (5, 10, 20)})       # {5:11, 10:12, 20:5}
print({k: count_changes(s, 10, k) for k in (1.5, 2.0, 3.0)})  # {1.5:12, 2.0:12, 3.0:5}
```

posco

실습 8. 단일 설비 상태 추적 미니 통합

이동통계·변화점·변동성을 모아 한 설비 상태 요약

목표

이동통계·변화점·변동성을 한데 모아 한 설비의 상태를 요약

단계

· 이동평균·이동표준편차·변화점을 한 번에 계산

· 초반·후반 변동성과 변화점 개수를 요약

· 후반 변화점 비율로 상태를 한 문장으로 진단

예상 결과

초반/후반 변동성 0.011 / 0.000, 변화점 12개, 후반 변화점 비율 0.09

posco

실습 8. 단일 설비 상태 추적 미니 통합 정답

이동통계·변화점·변동성을 모아 한 설비 상태 요약

CODE · 실습 8

```
s, w = u2['컨베이어전류'], 10
u2['ma'] = s.rolling(w).mean(); u2['std'] = s.rolling(w).std()
resid = s - u2['ma']; thr = 2 * resid.std()
u2['chg'] = resid.abs() > thr
half = u2['시퀀스'].median()
print('초반/후반 변동성:', round(u2['std'].iloc[10:50].mean(), 4), round(u2['std'].iloc[150:].mean(), 4))  # 0.011 / 0.0
print('변화점:', int(u2['chg'].sum()), '후반 비율:', round(u2.loc[u2.시퀀스 > half, 'chg'].mean(), 2))  # 12 / 0.09
```